

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 1 — Lo stesso prompt su più modelli Claude

Lo stesso prompt su tre modelli

Lo stesso prompt su tre modelli

L’esercitazione M1-E01: obiettivo, tempo, due percorsi

- **Obiettivo**: scegliere un modello sul vostro compito, misurando — lo stesso prompt su **Haiku**, **Sonnet** e **Opus**, poi il confronto di risposte, tempi e token
- **Tempo**: quindici minuti, undici dei quali al terminale
- **Percorso A**, un vostro compito: scrivetelo in `prompt/mio.md`, con i dati sostituiti da **valori inventati** — niente clienti, password o configurazioni reali
- **Percorso B**, i progetti di esempio (dati inventati):`prompt/sviluppo.md` (una funzione PHP del gestionale), `prompt/sistemi.md` (un estratto del syslog della rete), `prompt/supporto.md` (l’e-mail di un cliente e la guida al tono di voce); la prova del docente è su `sistemi.md`
- **Che cosa osservare**: dove le risposte **differiscono**, non quanto sono lunghe; il tempo e i token di ciascun modello
- Nel repository delle esercitazioni, il `README.md` di `m1_fondamenti_llm/s1_fondamenti_llm/m1_e2_stesso_prompt_tre_modelli/M1-E01_stesso_prompt/`

Il comando

Tre chiamate, una per modello: al posto di sistemi.md, il vostro prompt

```
p=prompt/sistemi.md
mkdir -p risposte
for m in haiku sonnet opus; do
  claude -p "$(cat "$p")" --model "$m" --effort low --tools "" \
    --system-prompt "Rispondi in italiano." --strict-mcp-config \
    --output-format json > "risposte/$m.json"
done
```

- `-p`: stampa la risposta ed esce, senza sessione interattiva
- `--model`: un alias, `haiku`, `sonnet` o `opus`, o il nome completo
- `--effort low`: il livello di effort più basso, da low a max
- `--tools` vuoto: nessuno strumento integrato
- `--system-prompt`: l'istruzione di sistema
- `--strict-mcp-config`: nessun server MCP (gli strumenti esterni, nel Modulo 4), quindi nemmeno i connettori del vostro account claude.ai
- `--output-format json`: un JSON con la risposta e i metadati, uno per modello

Che cosa c’è nel JSON

risposte/opus.json della prova del docente: un estratto

```
{
  "session_id": "94e8fb1e-62b1-4430-affa-b31f87763baf",
  "total_cost_usd": 0.02018799999999998,
  "usage": {
    "input_tokens": 2,
    "cache_creation_input_tokens": 1540,
    "cache_read_input_tokens": 0,
    "output_tokens": 393, ... },
  "modelUsage": { "claude-opus-5-5": { ... } }, ...
  "result": "1. **Cosa è successo:** alle 10:12:31 la ...", ...
  "duration_ms": 5452, ...
}
```

- `result`: la risposta; `duration_ms`: la durata, in millisecondi; `modelUsage`: il modello che ha risposto
- `usage`: i token della conversazione principale — in entrata tre campi, da sommare; in uscita `output_tokens`
- € `total_cost_usd`: una **stima** lato client, non una fattura — sul piano Pro l’uso è incluso nell’abbonamento

Al lavoro

I passi del README di M1-E01, in breve

1. `tools/prepara_esercizio.sh M1-E01`, poi `cd ~/corso-claude-code/M1-E01`
2. Scegliete il prompt, percorso A o B, e leggetelo
3. Le tre chiamate del comando: tre file in `risposte/`
4. `claude --model sonnet` nello spazio di lavoro, e chiedete:

```
(sessione su Sonnet, nello spazio di lavoro)

Mostrami, uno dopo l'altro, il campo result di
risposte/haiku.json, risposte/sonnet.json e risposte/opus.json,
senza commentarli.
```

5. Incollate il testo di `consegna_confronto.txt`: Claude scrive in `confronto.md` i secondi e i token di ogni modello — se vi chiede il permesso di modificare il file, concedetelo
6. Scrivete voi, in `confronto.md`, il prompt usato, che cosa cambia nella risposta, il modello che scegliereste e perché
7. `/exit`, poi `cd -` per tornare nella cartella `esercitazioni`, e `tools/verifica.sh M1-E01`

Checkpoint: i tre file in `risposte/` a metà tempo; alla fine la verifica esce con 0